

PlayCover / PlayTools 直接渲染捕获开发方案

面向《明日方舟》Unity Metal 渲染目标的 pre-compositor 图像接口

代码调查报告

2026 年 9 月 1 日

摘要

本方案针对 PlayCover 中现有窗口截图受逻辑尺寸、虚拟 native scale、Retina backing scale 和窗口装饰影响的问题，设计一条只适配 `com.hypergryph.arknights` 的直接渲染捕获链。目标是在被 PlayTools 注入的目标进程内，取得 Unity/Metal 提交给 CoreAnimation 之前的 color target，而不是再次从 macOS 窗口合成结果反向采样。

方案明确区分现状、设计和验证状态：当前固定 PlayTools revision 只有 ScreenCaptureKit/CGWindow 窗口采集；Arknights 的 UnityFramework 虽然包含 Unity Metal 和显示表面线索，但尚未证明具体运行时 drawable 属性或 hook ABI。推荐先部署不复制像素的运行时 probe，再在确认条件满足后，于原始 `presentDrawable`：提交前编码 GPU copy，使用有界环形缓冲异步 readback，最后通过新增 MaaTools 命令传输带完整元数据的原始帧。旧 `SCRN/BGR\1` 保持不变，失败时回退到原窗口采集。

目录

1 执行摘要	4
1.1 问题定义	4
1.2 结论与可行性等级	4
1.3 推荐实施顺序	4
2 范围、依据与非目标	5
2.1 目标对象和版本边界	5
2.2 证据标签	5
2.3 本方案包含的目标	5
2.4 明确的非目标	5
3 现状基线与关键证据	6
3.1 PlayCover 主进程的职责	6
3.2 现有 PlayTools 截图链	6
3.3 Arknights 的 Unity Metal 线索	7

3.4	尺寸边界	7
4	总体架构	8
4.1	进程和模块边界	8
4.2	模块职责	8
4.3	双层 hook 策略	8
5	阶段 M0: 运行时 Probe	9
5.1	目的和原则	9
5.2	采集字段	9
5.3	探测顺序	10
5.4	M0 判定表	10
6	阶段 M1: Metal 捕获核心	10
6.1	捕获边界	10
6.2	候选 hook 的优先级	11
6.3	资源模型	11
6.4	GPU copy 和 readback	11
6.5	framebuffer-only 分支	12
6.6	MSAA、resolve 和 memoryless	12
6.7	像素格式与色彩	12
6.8	方向和旋转	13
6.9	overlay 语义	13
7	阶段 M2: MaaTools 协议和客户端	13
7.1	兼容原则	13
7.2	建议的 RND0 响应	14
7.3	服务器状态机	14
7.4	MaaCore 适配	15
8	阶段 M3: 配置、gate 与回退	15
8.1	Arknights gate	15
8.2	开关策略	15
8.3	回退条件	16
8.4	网络和权限边界	16

9 阶段 M4：构建、注入和部署	16
9.1 需要修改的外部工程	16
9.2 PlayCover host 的变更边界	17
9.3 Carthage 和 Catalyst 产物	17
9.4 签名和部署检查	17
10 M5：测试与验收矩阵	18
10.1 M0 Probe 测试	18
10.2 M1 GPU copy 测试	18
10.3 协议测试	18
10.4 显示和几何测试	19
10.5 性能和稳定性验收	19
11 风险、取舍与回滚	20
11.1 技术风险	20
11.2 安全和合规风险	20
11.3 回滚方案	20
12 里程碑与交付物	20
13 实施文件清单	21
13.1 第一阶段实际需要新增或修改的文件	21
13.2 本次文档交付不修改的内容	21
14 诊断日志和协议验收模板	22
14.1 建议的 probe 日志	22
14.2 协议不变量	22
15 最终建议	22
A 证据索引	23
B 当前状态声明	23

1 执行摘要

1.1 问题定义

当前 MaaTools 的截图链路是：

```
Unity / Metal render target
-> PlayTools virtual screen and NSWindow
-> ScreenCaptureKit or CGWindowListCreateImage
-> title-bar crop and CGContext/vImage conversion
-> MaaTools TCP response
```

该链路适合获得“用户在窗口中看到的图像”，但不适合作为稳定的渲染帧接口。PlayCover 的逻辑窗口尺寸、PlayTools 伪造的 UIKit native 尺寸、macOS 窗口的 backing pixels，以及图像采集 API 的实际返回像素并不一定相同。对 720p 做二次转换时，尺寸、裁剪、方向和通道语义容易发生漂移。

本方案把目标改为：

```
Unity Metal render target
-> in-process render tap
-> GPU copy to bounded capture slots
-> asynchronous readback
-> MaaTools direct-render response
```

1.2 结论与可行性等级

项目	判断
针对 Arknights 获取	[已证实] 技术上可行，但必须先完成运行时 probe。
Unity/Metal color target	
绕过 NSWindow、	[已证实] 设计上可行。
ScreenCaptureKit、CGWindow	
和 Retina backing scale	
仅修改 PlayCover 主程序	不可行；渲染对象位于注入后的目标应用进程。
复用当前 PlayTools 不做重建	不可行；固定 revision 没有 Metal capture 模块。
使用 <code>presentDrawable</code> ：直接	不可行；需要在 <code>present</code> 前 GPU copy，完成后异步读取。
CPU 读像素	
准确输出真实 render target 尺寸	可行；尺寸必须来自 drawable/texture 元数据。
保证包含所有	不保证；drawable 位于窗口合成之前。
CoreAnimation/UIKit overlay	
保持旧 MAA 客户端可用	可行；保留旧命令，新增 direct-render 命令。

1.3 推荐实施顺序

1. 固定目标应用 gate，并建立不修改像素的运行时诊断 probe。

2. 观察 Unity 动态加载、CAMetalLayer、drawable 和实际 present selector。
3. 记录 framebufferOnly、尺寸、格式、采样数和存储模式。
4. 只复制一帧到自有 GPU/CPU 缓冲，验证同步和像素完整性。
5. 新增 MaaTools direct-render 命令，保留旧截图命令。
6. 生成 arm64/Catalyst 产物，经过 PlayCover 现有注入和重签名链部署。
7. 执行窗口、旋转、resize、后台、性能、完整性和回退测试。

2 范围、依据与非目标

2.1 目标对象和版本边界

本方案只针对以下本机测试对象：

字段	已知值
Bundle identifier	com.hypergryph.arknights
应用版本	2.7.61, build 59
主架构	arm64
引擎	Unity 2021.3.39f1, IL2CPP
图形后端证据	直接链接 Metal/OpenGL ES; 运行时加载 AGXMetal 和 IOSurface
PlayCover	MaaTools 分支, HEAD 63cf605
PlayTools 依赖	hguandl/PlayTools MaaTools 分支, revision 7aa1977
当前配置	内容目标 1280×720、customScaler=1、MaaTools 端口 1717

PlayTools 的固定 revision 和 Arknights 的 UnityFramework 都可能随版本变化。本文的 Unity 内部类名和 selector 只能作为当前版本的适配依据，不能作为通用 Unity API 或稳定 ABI。

2.2 证据标签

- **[已证实]**：已经由仓库、固定依赖、IPA、已安装应用或已有运行时观察确认。
- **[待验证]**：需要新增 probe、构建产物或真实运行测试才能确认。
- **[建议]**：本方案建议采用的工程做法，不表示当前代码已经具备该能力。

2.3 本方案包含的目标

- 在目标进程内获取 Unity/Metal pre-compositor color target。
- 让输出携带真实像素宽高、行距、格式、方向、帧号和状态。
- 消除窗口标题栏、AppKit frame 和 macOS backing scale 对原始帧尺寸的影响。
- 对 Arknights 做严格 gate，并在条件不满足时安全回退。
- 保持既有 MaaTools 命令的兼容性。

2.4 明确的非目标

- 不把本方案扩展为所有 PlayCover 应用的通用渲染 hook。

- 不修改 UnityFramework 内部二进制或重新编译 Arknights 的游戏逻辑。
- 不获取 macOS 桌面、其他应用窗口或屏幕内容。
- 不绕过应用完整性、反作弊或服务端安全策略；完整性 SDK 只作为兼容性风险进行测试。
- 不在本阶段移除或改变现有窗口截图后端。

3 现状基线与关键证据

3.1 PlayCover 主进程的职责

PlayCover 主程序不持有游戏的 Metal command buffer。它负责将 PlayTools 安装到用户 framework 目录、向目标 Mach-O 添加 load command、复制 AKInterface.bundle、签名并启动应用。

- PlayCover/PlayCover/Model/PlayApp.swift:64–116: 同步设置、检查 PlayTools、安装插件和启动目标应用。
- PlayCover/PlayCover/Utils/PlayTools.swift:46–91: 系统 framework 安装、Mach-O 注入和签名。
- PlayCover/PlayCover/Utils/PlayTools.swift:94–125: 将 AKInterface.bundle 复制进目标应用并重新签名。
- PlayCover/PlayCover/Utils/PlayTools.swift:150–187: 导出 IPA 时复制 PlayTools.dylib 并注入 @executable_path/Frameworks 路径。

PlayCover 在 PlayCover/PlayCover/Model/PlayApp.swift:124–163 清除 DYLD_*、METAL_* 和 MTLCaptureEnabled 等环境变量。这是启动环境管理，不是截图实现。因此 direct-render 不能依赖 Metal 调试环境变量注入，应该作为 PlayTools 内部代码随静态 load command 进入目标进程。

3.2 现有 PlayTools 截图链

固定 revision 的外部源码中，MaaTools.swift 的截图处理调用 AKInterface.windowImage()。AKPlugin.swift 再通过 macOS 14.4 以上的 ScreenCaptureKit 或旧版 CGWindow API 获取窗口图像，随后由 MaaTools 进行标题栏裁剪和栅格化。

接口	当前语义
SCRN	返回长度和 4 bytes/pixel 的原始 sRGB 缓冲；宽高由独立的 SIZE 提供。
BGR\1	返回裁剪后宽度、高度、长度和 3 bytes/pixel 缓冲；实现使用 RGBA8888 到 RGB888 转换，命令名不能单独证明通道顺序。
SIZE	返回缓存的窗口内容尺寸，不是 Metal drawable 的可靠像素尺寸。

现有协议的握手和请求帧为：

```
client -> MAA\0
```

```
server -> OKAY
request -> u16be payload_length || payload
payload[0..3] = command magic
```

3.3 Arknights 的 Unity Metal 线索

已安装 Arknights 的 UnityFramework 中可观察到以下静态线索：

```
Unity 2021.3.39f1
GfxDeviceMetal
UnityDisplaySurfaceMTL
CAMetalLayer
nextDrawable
presentDrawable:
DisplayConnection.present
DisplayManager.present
UnityRenderingView.layerClass
UnityView.recreateRenderingSurface
MTLCommandBuffer
MTLTexture
setFramebufferOnly:
```

运行中的目标进程还加载了 `AGXMetalG17G.bundle`、`IOSurface` 和 `IOAccelerator`。上述事实支持 Metal 路径的研究，但不等于已经证明当前每一帧都由同一 selector 提交，也不等于已获得可复制的 drawable。具体调用频率、参数编码和对象生命周期必须由 probe 确认。

3.4 尺寸边界

当前 PlayCover 配置的内容目标为 1280×720 ，窗口外部 bounds 观察到约为 1280×748 ，显示器 backing scale 为 2.0。应用自己的 Unity 偏好中还出现宽高为 0、全屏和无边框的设置，意味着 Unity 可根据显示目标动态决定 render size。

方案中定义以下尺寸，避免混用：

符号	含义
W_0, H_0	PlayCover/PlayTools 的逻辑窗口内容尺寸，接近 UIKit points。
S	PlayTools 伪造给应用的 native scale 或 custom scaler。
B	macOS 窗口实际 backing scale。
W_r, H_r	Unity/Metal 实际 render target 的 pixels，必须从 drawable/texture 读取。
T	客户端可选的统一输出尺寸，例如 1280×720 。

direct-render 的源尺寸是 W_r, H_r ，而不是 W_0, H_0 、 $W_0 \times B$ 或窗口 frame。

4 总体架构

4.1 进程和模块边界

```
PlayCover host process
+-- AppSettings / install / injection / signing
|-- launch target application
|
v
Arknight's target process
+-- PlayTools.dylib / framework
|   +-- existing PlayScreen / PlayInput
|   +-- existing MaaTools TCP server
|   |-- new Arknight'sMetalCapture
+-- AKInterface.bundle
+-- UnityFramework
|   +-- UnityDisplaySurfaceMTL
|   +-- CAMetalLayer
|   |-- Metal command queue / command buffer
|
v
MaaCore client
|-- direct-render command or legacy SCRNBGR\1
```

4.2 模块职责

模块	职责与边界
Arknight'sMetalCapture	bundle gate、动态加载后安装 hook、probe、资源创建、GPU copy、slot 管理和异步 readback。不访问窗口，不执行网络发送。
UnitySurfaceProbe	只记录 Unity surface/layer/drawable 的运行时代数据，首阶段不复制像素。
MaaTools.swift	保存最新完成帧，处理 direct-render 命令和响应序列化；继续提供旧命令。
AKPlugin.swift	保留现有窗口采集和完整窗口 fallback；不承担 Metal command buffer 同步。
PlayCover host	默认负责现有设置、注入、复制和签名；只有增加用户可见开关时才扩展设置模型/UI。
MaaCore controller	解析新响应、检查元数据和 stride，旧协议保持原解析逻辑。

4.3 双层 hook 策略

UnityFramework 不一定由 Arknight's 主 Mach-O 直接链接，可能在启动后动态加载。因此不能只在 PlayLoader constructor 中立即查找 DisplayConnection。

[\[建议\]](#)采用两层策略：

1. 在 PlayTools constructor 中只完成目标 gate、状态初始化和通用的低侵入观察准备。

2. 监听 UnityFramework 动态加载，或使用受控的短周期延迟任务等待其进入地址空间。
3. Unity 类出现后，以 Objective-C runtime 的 selector 和 method encoding 验证 UnityRenderingView、DisplayConnection 或 DisplayManager 适配点。
4. 同时保留 CAMetalLayer.nextDrawable 诊断路径，作为版本专用 Unity hook 失效时的观察和兜底入口。
5. 在所有 hook 中使用 dispatch_once、递归保护和卸载/重建状态，避免重复安装。

不建议使用 UnityFramework stripped 后的固定地址作为第一版方案。若未来必须使用地址特征，应把 UUID、版本和特征校验写成独立的适配层，并在不匹配时关闭 direct-render，而不是继续调用未知地址。

5 阶段 M0：运行时 Probe

5.1 目的和原则

M0 的目的不是拿到图像，而是回答 direct-render 是否具备实施条件。Probe 必须：

- 只读对象属性，不修改 layer、drawable 或 command buffer；
- 不调用 getBytes、waitUntilCompleted 或同步网络；
- 对重复帧限频，只在对象、尺寸、格式或状态改变时记录；
- 只对目标 bundle 生效；
- 任何 selector、encoding 或对象类型不匹配都立即熔断。

5.2 采集字段

每次 layer/drawable 变化至少记录：

```
process bundle identifier
UnityFramework UUID and version evidence
layer class and pointer identity
layer.contentsScale
layer.drawableSize.width / height
layer.pixelFormat
layer.framebufferOnly
drawable class and texture class
texture.width / height
texture.pixelFormat
texture.sampleCount
texture.storageMode
texture.usage
command-buffer concrete class
present selector and Objective-C type encoding
thread identity and command-buffer status
```

日志不应记录账号、AppKey、网络请求正文或完整帧内容。指针地址仅用于当前进程内诊断，不应作为持久配置或协议字段。

5.3 探测顺序

1. 确认 `Bundle.main.bundleIdentifier` 等于目标 bundle ID。
2. 等待 `UnityFramework` 映像出现；记录 UUID 和路径，只作版本校验。
3. 观察 `UnityRenderingView.layerClass` 或实际 view 的 layer 类型。
4. 在 `nextDrawable` 返回后读取 layer、drawable 和 texture 的只读元数据。
5. 观察 Unity surface 的 `present` 相关方法，确认是否每帧触发以及运行线程。
6. 在启动、登录后场景、进入战斗、窗口 `resize` 和前后台切换时重复记录。

5.4 M0 判定表

观察结果	后续动作
稳定出现主 <code>CAMetalLayer</code> ，drawable 尺寸非零，selector 可验证	进入 M1，先做单帧 GPU copy。
只有 Unity surface 方法，没有可观察 layer	以 Unity surface 的 <code>present</code> 边界继续探测，不假设参数布局。
有 layer 但没有 <code>present</code> 调用	检查 <code>present</code> 变体或改为观察 <code>command buffer</code> ；不在 <code>nextDrawable</code> 处读像素。
<code>framebufferOnly=NO</code> 且 texture 为单采样	可尝试 <code>drawable blit</code> 。
<code>framebufferOnly=YES</code>	进入 <code>resolve/intermediate</code> 分支，暂不承诺 <code>drawable</code> 可复制。
出现多个同尺寸 layer 或 layer 频繁重建	以生命周期和主 surface 识别为前置条件，未识别时关闭直采。
任何 selector encoding 不匹配或发生异常	记录原因并回退窗口采集。

6 阶段 M1: Metal 捕获核心

6.1 捕获边界

推荐的时间顺序是：

```
Unity encodes render commands
-> final resolve or blit
-> capture copy encoded into the same command buffer
-> original presentDrawable call
-> command buffer commit and completion
-> background queue reads capture slot
```

`presentDrawable`：是一个提交边界候选，而不是 CPU 读回 API。hook 中应先确认 `command buffer` 仍处于 `encoding` 状态，然后编码对自有资源的 `copy`，最后调用原始 `present` 实现。不得在 hook 中等待 GPU，也不得在渲染线程同步发送 TCP。

6.2 候选 hook 的优先级

1. Unity 版本专用的 `DisplayConnection.present` 或 `DisplayManager.present`: 若 runtime encoding 可验证, 优先使用, 因为更接近 Unity 自己的 surface 管理。
2. `CAMetalLayer.nextDrawable`: 用于关联主 layer、记录 drawable 属性和捕获 resize; 它本身不是最终帧完成点。
3. `MTLCommandBuffer` concrete class 的 `presentDrawable::` 用于在 command buffer 中追加 copy; 具体 class 由运行时 Metal 驱动决定。
4. Unity 内部 stripped C++ 地址或指令特征: 仅作为后续版本专用适配, 不作为首选。

6.3 资源模型

每个 capture slot 至少包含:

```
slot state: free / encoding / inFlight / ready / dropped
width, height, bytesPerRow
pixel format and color-space tag
MTLBuffer or staging texture
frame ID and monotonic timestamp
orientation and rotation
MTLCommandBuffer completion status
```

[建议]使用两个或三个有界 slot。渲染线程只取得 free slot 并编码 copy; GPU 完成后由 completion handler 将 slot 标记为 ready; MaaTools 读取时只取最新 ready slot, 并将过期 ready slot 丢弃。客户端慢时不应反向阻塞 render thread。

6.4 GPU copy 和 readback

优先使用以下路径:

1. 创建与源尺寸一致的共享或 staging 资源;
2. 按实际 row pitch 对齐 buffer;
3. 在原 command buffer 中追加 texture-to-buffer 或 texture-to-texture copy;
4. 调用原始 present;
5. 在 completion handler 中确认 status/error;
6. 将 ready slot 发布给专用串行 readback 队列;
7. 读取前再次检查 slot 的 frame ID 和尺寸, 防止 resize 期间误用旧资源。

不允许以下实现:

- 在 `presentDrawable`: 内调用同步 `waitUntilCompleted`;
- 直接对 framebuffer-only 或 private texture 调用未经验证的 CPU readback;
- 将 drawable 本身长期 retain 作为帧缓存;
- 每一帧额外调用 `nextDrawable`;
- 在 Metal 内部线程上进行阻塞式 socket 写入。

6.5 framebuffer-only 分支

`CAMetalLayer.framebufferOnly` 是第一项硬条件。其处理决策如下：

状态	处理
NO	检查 texture usage、storage、sampleCount 后尝试同 command buffer blit。
YES, 可访问已 resolve intermediate target	捕获 resolve 后的 intermediate target。
YES, 无可访问中间目标	仅在 probe 环境测试创建前设为 NO；若改变渲染行为则关闭直采。
未知或对象已重建	不复制像素，等待下一次 probe；必要时回退窗口采集。

不能仅在 drawable 创建后修改 `framebufferOnly` 就假定已有 drawable 的使用限制已经改变。强制设为 NO 可能改变性能和 Unity 行为，必须作为 Arknights-only 的实验开关，不应默认对所有应用执行。

6.6 MSAA、resolve 和 memoryless

若 Unity 的渲染顺序为：

```
MSAA render target -> resolve -> single-sample drawable -> present
```

在 drawable 可复制的前提下，捕获 drawable 通常能得到已 resolve 的最终颜色。以下情况必须由 probe 或 Metal validation 结果确认：

- source texture 的 `sampleCount` 是否为 1；
- resolve 是否发生在当前 command buffer 的 copy 之前；
- render pass 是否使用 memoryless attachment；
- store action 是否保留 resolve 结果；
- 后处理、TAA、色调映射是否在另一个 command buffer 中追加；
- Unity 是否先绘制到 RenderTexture，再将最终结果 blit 到 drawable。

如果源是多采样纹理，必须先 resolve，不能把它按普通 2D 像素缓冲解释。如果 render pass 使用 discard 或 memoryless，过了该 render pass 后可能已没有可恢复的内容，此时 capture 点必须前移到 resolve 阶段。

6.7 像素格式与色彩

协议不得假定所有帧都是 4 bytes/pixel。第一版实现可优先支持实际 probe 得到的常见格式，例如 BGRA8Unorm、BGRA8Unorm_sRGB、RGBA8Unorm 和 RGBA8Unorm_sRGB；其他格式明确返回 unsupported 状态并回退。

每种支持格式应定义：

- Metal pixel format 到协议枚举的映射；
- bytes per pixel 和 row pitch；
- R/G/B/A 通道排列；
- sRGB/linear/HDR 标识；

- 客户端是否需要转换为 BGR888 或 RGBA8888。

Arknights 的 boot 配置包含 `hdr-display-enabled=0`，这使 SDR 8-bit 路径更可能成立，但不能代替运行时检查。若出现 `RGBA16Float` 等 HDR 格式，应先转成明确的 SDR 输出，或返回不支持状态；不能把 `float16` 数据伪装成 8-bit 数据。

颜色通道必须使用红、绿、蓝色块探针验证。命令名 `BGR\1` 不足以证明实际字节顺序，因为现有实现的 `vImage` 调用语义与命令名并不完全一致。

6.8 方向和旋转

Metal texture 通常不携带用户界面方向语义，输出方向需要单独记录。每一帧元数据应包含：

```
orientation: portrait / landscape-left / landscape-right / unknown
rotation: 0 / 90 / 180 / 270
mirrored: false or true
content origin: top-left or bottom-left
```

MaaTools 和 MaaCore 应把“像素内存方向”和“触控坐标方向”分开处理。不要把 `PlayCover` 的 `displayRotation` 直接当作 Metal texture 的旋转；必须用已知角落文字或测试图验证。

6.9 overlay 语义

`direct-render` 返回的是 Unity/Metal drawable，不是 macOS 最终合成画面。它通常不包含：

- macOS 标题栏、边框和阴影；
- 位于 Unity view 上方的独立 UIKit/CoreAnimation layer；
- 系统键盘或独立窗口；
- PlayTools 创建的非 drawable 浮层。

已经绘制进 Unity 最终 render pass 的游戏 UI 会包含在内。方案应将该语义写入协议能力说明，并保留现有窗口采集作为“完整窗口”后端。

7 阶段 M2：MaaTools 协议和客户端

7.1 兼容原则

旧客户端不认识未知 magic，且旧命令已经被使用。因此：

- 不改变 `SCRN`、`BGR\1`、`SIZE` 的已有布局和语义；
- 新增独立 `direct-render` 命令，例如 `RND0`；
- 只有支持该命令的客户端才启用直采；
- `direct-render` 失败时，客户端可以继续请求旧命令；
- `VERN=3` 不应被偷偷改成另一种旧命令语义；如需能力发现，增加显式能力查询或在新响应中返回能力状态。

7.2 建议的 RND0 响应

请求可以沿用现有握手和 16-bit 大端长度帧：

```
connect:
    client  -> MAA\0
    server  -> OKAY

request:
    u16be payload_length || "RND0" || request_flags
```

建议响应使用固定大端序头部，再跟随原始字节。具体版本应在实现前冻结并写入单元测试：

字段	含义
magic	响应 magic，例如 RND0。
version:u16	direct-render 响应版本，与旧 MaaTools 协议版本分开。
status:u16	ready、no-frame、unsupported-format、not-target、temporarily-disabled 等。
frameID:u64	单调递增的帧序号。
timestamp:u64	单调时钟纳秒或约定的时间单位。
width:u32	render target 像素宽度。
height:u32	render target 像素高度。
bytesPerRow:u32	实际行距，不能默认等于 width×BPP。
pixelFormat:u32	协议枚举，不直接暴露平台私有数值。
orientation:u16	方向枚举。
rotation:u16	旋转枚举。
dataLength:u32	后续像素字节数。
data	按上述格式和 row pitch 编码的像素数据。

若状态不是 ready，响应仍应带有可解析的 status 和错误原因，不能让客户端把空数据当作有效的零尺寸帧。长度计算必须使用 64-bit 中间值并在写入 u32 前检查上限。

7.3 服务器状态机

```
Disabled
-> TargetVerified
-> ProbeReady
-> CaptureArmed
-> FrameInFlight
-> FrameReady
-> ClientRead

任何 selector mismatch / GPU error / resize race
-> TemporarilyDisabled
-> legacy window capture fallback
```

MaaTools 不应为了响应一次请求临时创建 drawable，也不应等待下一帧无限阻塞连接。没有 ready 帧时返回明确状态；客户端可短暂重试或请求旧后端。

7.4 MaaCore 适配

若采用新命令，MaaCore 的 PlayTools controller 需要新增响应解析和能力选择。已知的外部客户端控制器位置为：

```
maa/src/MaaCore/Controller/PlayToolsController.h  
maa/src/MaaCore/Controller/PlayToolsController.cpp
```

客户端必须：

1. 按大端序读取固定头部；
2. 校验 status、width、height、bytesPerRow 和 dataLength；
3. 检查 dataLength 是否覆盖所有行距；
4. 根据 pixelFormat 解码，而不是由命令名猜通道；
5. 根据 orientation/rotation 做显示或识别坐标变换；
6. 在 direct-render 不支持时自动使用旧 SCRNB 或 BGR\1。

8 阶段 M3：配置、gate 与回退

8.1 Arknights gate

第一版必须在目标进程内执行硬 gate：

```
Bundle.main.bundleIdentifier == "com.hypergryph.arknights"
```

可选的第二层校验包括 UnityFramework UUID、Unity 版本字符串和目标 framework 路径。任何校验失败都关闭 direct-render，不影响 PlayTools 的普通输入和旧截图服务。

8.2 开关策略

[建议]第一版不要立即增加复杂的 PlayCover UI。可以先将 direct-render 作为 MaaTools 内部的 Arknights-only 能力：

- 旧 maaTools 仍决定 MaaTools listener 是否启动；
- 新能力只有在 Arknights gate 和 probe 同时成功后自动 armed；
- 客户端通过新命令请求，未请求时不做像素 copy；
- 发生失败时只记录一次原因并回退。

若后续需要用户可见的“Metal direct capture”开关，再同步修改：

- PlayCover/PlayCover/Model/AppSettings.swift:10-56：新增可编码设置及默认值；
- PlayCover/PlayCover/Views/AppSettingsView.swift:576-586：新增开关和说明；
- 外部 PlayTools 的 PlaySettings.swift：读取同一个 plist 字段；
- 本地化资源：解释该模式不包含独立 CoreAnimation overlay。

8.3 回退条件

以下任一条件出现时，direct-render 应暂时禁用并回到 `AKInterface.windowImage()`：

- bundle、Unity UUID 或 selector encoding 不匹配；
- drawable 尺寸为零或尺寸变化尚未完成资源重建；
- `framebufferOnly` 阻止复制且没有可用 resolve target；
- `sampleCount`、`pixelFormat` 或 `storage mode` 不在支持矩阵；
- `command buffer status/error` 表示 copy 或 present 失败；
- ring buffer 连续耗尽、GPU 延迟超过阈值或读取数据校验失败；
- 应用进入后台、Unity layer 被重建或目标进程退出。

回退不应删除 hook 或修改 Unity 层；只需停止 armed 状态，释放 capture slot，并让旧窗口采集继续工作。

8.4 网络和权限边界

direct-render 访问的是目标进程自己的 Metal 资源，不调用 `ScreenCaptureKit`，因此理论上不需要 Screen Recording TCC。现有 MaaTools 的网络服务仍需单独处理：

- listener 明确绑定 127.0.0.1，而不是依赖标题中的 `localhost`；
- 对新命令设置请求长度、连接超时和响应大小限制；
- 截图能力与 TOUCH/TERM 控制能力分离授权；
- 不将像素帧写入持久日志；
- 端口冲突或非预期来源时返回可诊断错误。

9 阶段 M4：构建、注入和部署

9.1 需要修改的外部工程

当前 PlayTools 源码不在 PlayCover checkout 中，固定 revision 的实际修改应发生在外部 PlayTools 工程：

文件/target	计划变更
PlayTools target	新增 Metal capture 源文件，加入 Metal/QuartzCore 依赖，保留现有 MaaTools。
MaaTools.swift	新命令、最新帧状态、序列化、fallback 和 listener 生命周期。
AKPlugin.swift	继续提供窗口采集；若 probe 需要 layer 发现，可增加最小适配桥接，但不复制渲染帧。
PlayCover.swift/PlayLoader.m	目标 gate、动态加载后初始化或 probe 启动入口。
PlayTools.xcodeproj/project.pbxproj	新源文件、framework link、编译条件和 arm64/Catalyst target。
PlaySettings.swift	只有新增 direct-render 配置字段时才修改。

9.2 PlayCover host 的变更边界

如果仍复用用户级 `PlayTools.framework` 路径, `PlayCover/PlayCover/Utils/PlayTools.swift:46-187` 的注入、复制和签名逻辑原则上可以复用。当前 host 不需要知道 Metal texture, 也不需要访问目标进程内存。

只有以下情况需要修改 PlayCover host:

- 新增独立 dylib, 导致需要额外复制或注入路径;
- 新增用户设置开关, 需要修改设置模型和 UI;
- 新 framework 资源布局改变, 导致 `installPluginInIPA` 复制规则改变;
- 新协议能力需要 host 在启动前传递非 plist 参数。

不建议通过 `DYLD_INSERT_LIBRARIES` 作为正式方案, 因为 PlayCover 会清理 DYLD 环境变量, 而且环境变量路径会增加首帧竞态和签名复杂度。

9.3 Carthage 和 Catalyst 产物

PlayCover 的 `PlayCover/PlayCover.xcodeproj/project.pbxproj:578-597` 构建脚本当前:

1. 从 `Carthage/Build/PlayTools.xcframework/ios-arm64` 读取 framework;
2. 复制 `AKInterface.bundle`;
3. 用 `vtool` 设置 `maccatalyst build version`;
4. 重新组织 `deep framework` 目录;
5. 对插件和 framework 签名。

开发顺序应为:

```
external PlayTools source
-> build arm64 iOS/Catalyst-compatible framework
-> verify Metal.framework and symbols/selectors
-> Carthage/Build/PlayTools.xcframework
-> PlayCover copy/vtool/rebundle
-> codesign nested plugin and framework
-> inject target Mach-O
-> codesign target application
-> launch Arknights and inspect logs
```

只有在确实改变了 PlayTools 来源 revision 后, 才更新 [L] `Cartfile.resolved`; 不能仅为生成本地测试产物而把未审计的依赖 revision 写入发布配置。

9.4 签名和部署检查

每次部署必须检查:

- PlayTools、AKInterface 和目标应用均为 arm64;
- Metal/QuartzCore load command 在最终产物中存在;
- bundle gate 代码进入目标进程, 而不是只进入 PlayCover host;

- nested bundle 先签、外层 framework 后签、目标 app 最后签；
- 复制资源前清理会导致 codesign 失败的扩展属性；
- 目标 Mach-O 的 PlayTools load command 仍然存在且路径有效；
- 替换现有用户级 framework 后重新启动应用，不复用旧进程。

10 M5：测试与验收矩阵

10.1 M0 Probe 测试

场景	验证内容	通过标准
启动首帧	UnityFramework 动态加载和 hook 安装时序	不崩溃，能在首个稳定 frame 前观察对象
普通主界面	layer、drawable、present 关联	主 layer 的属性稳定，present 连续触发
进入战斗	渲染 target 和后处理变化	尺寸/格式变化被记录，不使用旧 slot
窗口 resize	drawableSize、layer 重建	资源重建，无旧尺寸帧混入
前后台切换	command buffer 状态和 layer 生命周期	暂停/恢复可识别，失败可回退
非目标应用	bundle gate	不安装或不执行 Arknights capture
selector 不匹配	encoding 和版本校验	禁用直采并记录一次原因

10.2 M1 GPU copy 测试

- 单帧 copy 后检查非零长度、尺寸和 frame ID；
- 检查 GPU validation 是否报告 framebuffer-only、usage 或 encoder 错误；
- 检查 completion handler 后再读取，禁止提前读取；
- 连续运行至少数百帧，确认没有 drawable starvation、死锁或明显卡顿；
- 使用色块/角落文字验证通道顺序、行距、上下方向和旋转；
- 在 resize 后确认每个 slot 的容量和 bytesPerRow 已更新。

10.3 协议测试

测试组	内容
握手与端序	MAA\0/OKAY、u16be 请求长度、所有 u32/u64 大端字段。
边界长度	零帧、最大允许尺寸、dataLength 溢出、半包、断开和重复请求。
格式校验	width、height、stride、pixelFormat、dataLength 一致性。
能力和失败	not-target、no-frame、unsupported-format、temporarily-disabled。
旧兼容	旧客户端继续使用 SCRN/BGR\1；新客户端可回退旧命令。
并发	多次请求、慢客户端、断线重连、帧丢弃和只保留最新帧。
控制隔离	direct-render 不扩大 TOUCH/TERM 权限，listener 只绑定 loop-back。

10.4 显示和几何测试

- 逻辑窗口 1280×720, macOS backing scale 1 和 2;
- 窗口化、全屏、borderless 和标题栏显示;
- 16:9、4:3 和可调整窗口;
- displayRotation 0、90、180、270 和 inverse screen values;
- resize 过程中连续请求帧;
- Unity UI、可能的 UIKit overlay 与窗口采集结果对照;
- 统一输出 1280×720 时分别验证 aspect-fit、aspect-fill 和黑边 validRect。

10.5 性能和稳定性验收

建议记录以下指标，而不是只观察“能否截图”：

```
capture request latency
GPU copy duration if available
command-buffer completion latency
readback queue depth
number of dropped frames
number of fallback transitions
render thread frame-time delta
resident memory of capture slots
TCP response throughput
```

第一版验收条件：

1. 目标应用在 probe 和 direct-render 开关打开时可正常启动和退出。
2. 不因 capture 产生 Metal validation error、死锁、drawable starvation 或可重复崩溃。
3. 输出尺寸来自真实 render target，且 stride、格式和数据长度可自洽验证。
4. 客户端能正确处理方向和通道，旧客户端不受影响。
5. resize、后台、Unity layer 重建和不支持格式都能回退。
6. 未请求 direct-render 时不进行不必要的每帧 GPU copy。
7. 非 Arknights 应用不安装、不执行该专用 capture 路径。

11 风险、取舍与回滚

11.1 技术风险

风险	影响	缓解
UnityFramework 动态加载	constructor 查找不到 Unity 类或错过首帧	动态加载后初始化，加通用 layer probe，版本不匹配则关闭。
私有 concrete Metal class	selector/实现随系统 GPU 变化	运行时 class/encoding 校验，避免固定地址，保留 fallback。
framebuffer-only	drawable 不可作为 copy source	先测实际值，优先捕获 resolve target，强制 NO 只做实验分支。
MSAA/memoryless	过晚捕获得不到有效颜色	确认 resolve/store 时序，必要时前移到 intermediate target。
GPU 同步	卡顿、旧帧、资源复用冲突	同 command buffer copy、有界 slot、completion 后 readback。
overlay 差异	direct frame 与用户所见画面不同	在协议中声明语义，保留窗口采集后端。
Unity/系统更新	私有 ABI 失效	UUID/版本 gate、probe、失败熔断和回退。

11.2 安全和合规风险

目标应用包含第三方完整性/安全组件。即使本方案只使用公开 Metal API，进程内 hook 仍可能改变调用路径和内存行为。测试应使用用户明确授权的本机应用、副本或测试账号；不应把本方案用于绕过完整性检查、修改游戏逻辑或扩大控制权限。

现有 MaaTools listener 还存在网络暴露和 TOUCH/TERM 混用风险。direct-render 不是解决这些问题的理由，部署时应将 loopback、认证、超时和控制命令隔离作为独立验收项。

11.3 回滚方案

回滚必须是可逆的：

1. 将 direct-render armed 状态置为 false；
2. 停止新命令的像素 copy 和 readback；
3. 释放 ring buffer，但不销毁现有 PlayScreen/PlayInput；
4. 让 AKInterface.windowImage() 继续服务 SCRN/BGR\1；
5. 若新 framework 影响启动，则恢复上一份已验证的 PlayTools framework 和目标应用签名产物。

不要使用不可恢复的二进制覆盖、强制 reset 或删除用户应用数据来处理失败部署。每次实验前应保留旧 framework hash、旧目标应用包和配置备份。

12 里程碑与交付物

里程碑	工作内容	退出条件
M0 Probe	Arknights gate、动态加载观察、layer/drawable/present 元数据日志	真实主 layer 和 present 路径可解释，或明确记录阻塞条件
M1 单帧 copy	资源创建、framebuffer-only 分支、同 command buffer blit、completion readback	单帧数据可校验，无 GPU 错误和渲染线程阻塞
M2 协议接入	RND0 头部冻结、服务端状态机、MaaCore 解析和 fallback	新旧客户端互不破坏，错误状态可解析
M3 构建部署	PlayTools framework、AKInterface、Carthage、Catalyst、注入和嵌套签名	全新启动应用并加载正确 UUID/版本的 PlayTools
M4 回归	resize、方向、格式、性能、后台、端口和完整性风险	验收矩阵通过，失败可自动回退
M5 可选产品化	host 设置/UI、能力说明、日志和发布脚本	用户可控开关、升级策略和回滚包齐全

13 实施文件清单

13.1 第一阶段实际需要新增或修改的文件

文件	计划
外部 PlayTools 的新 Metal capture 源文件	新增；负责 probe、hook、GPU copy、slot 和 readback。
外部 PlayTools MaaTools.swift	修改；增加 direct-render command、元数据和 fallback。
外部 PlayTools	修改或增加入口；处理动态加载后初始化。
PlayCover.swift/PlayLoader.m	
外部 PlayTools	修改；加入源码和 Metal framework link。
PlayTools.xcodeproj/project.pbxproj	
外部 MaaCore PlayToolsController.h/.cpp	若采用 RND0，则修改客户端解析和能力回退。
PlayCover/PlayCover/Model/AppSettings.swift:10-56	第一版可不改；只有需要 host 开关时修改。
PlayCover/PlayCover/Views/AppSettingsView.swift:576-586	第一版可不改；产品化开关时修改。
PlayCover/PlayCover/Utils/PlayTools.swift:46-187	同一 framework 路径复用时不改；独立 dylib/资源布局变化时才改。
PlayCover/PlayCover/Model/PlayApp.swift:124-163	不用环境变量方案时不改；禁止依赖其已清理的 capture 环境变量。

13.2 本次文档交付不修改的内容

本次任务只生成本方案的 LaTeX 和 PDF，不执行上述 PlayTools 实现。不会修改：

- PlayCover 应用源码；
- 外部 PlayTools 源码或二进制；

- MaaCore 源码；
- 用户提供的 IPA 和已安装 Arknights；
- PlayCover 的依赖锁定和签名配置。

14 诊断日志和协议验收模板

14.1 建议的 probe 日志

```
[MetalCapture] target=com.hypergryph.arknights
[MetalCapture] UnityFramework uuid=<validated>
[MetalCapture] layer=<class> drawable=1280x720 scale=1.0
[MetalCapture] pixelFormat=<enum> framebufferOnly=NO sampleCount=1
[MetalCapture] texture storage=<enum> usage=<mask> rowPitch=<value>
[MetalCapture] present selector=<validated> thread=<id>
[MetalCapture] capture armed; slots=3
[MetalCapture] frame=42 bytes=... status=ready
```

实际日志中不应把尖括号内容当作固定值，也不应写入账号或应用网络数据。对同一个进程，layer 地址和 frame ID 只用于诊断；重启后不得依赖地址稳定性。

14.2 协议不变量

实现和客户端测试应固定以下不变量：

1. 所有整数端序在协议文档中明确为大端序。
2. dataLength 不超过服务端允许的最大响应大小。
3. bytesPerRow 至少覆盖一行所需像素字节，并能覆盖全部数据长度。
4. width、height、pixelFormat、stride 和 dataLength 相互一致。
5. frame ID 单调递增；丢帧不要求连续，但不得回退。
6. ready 帧在完成回调之前不可被客户端读取。
7. status 非 ready 时，客户端不能把 data 当作有效图像。
8. direct-render 失败不改变旧命令的返回布局。

15 最终建议

本项目最稳妥的实施路线不是直接把现有 SCRn 改成 Metal 读取，而是新增一个具备明确语义的 direct-render 后端：

```
probe -> validate -> arm -> GPU copy -> async readback -> RND0
      |                                     |
      +----- failure / unsupported -----+
      |
      legacy window capture
```

在实际编写 hook 之前, M0 probe 是不可省略的验收门槛。特别是 `framebufferOnly`、`resolve` 时序、动态加载和 `concrete command-buffer class`, 无法由静态字符串或 `nm` 输出替代。

只要当前实例满足可复制 `drawable` 或可访问的 `resolve target`, Arknights-only 方案就具备继续开发的条件; 如果两个条件都不满足, 则应保留现有窗口采集, 而不是通过同步读回或未经验证的私有偏移强行实现。这样可以同时解决尺寸一致性问题、控制性能风险, 并使失败路径保持可逆。

A 证据索引

标签	文件或对象	支持的事实
[L]	PlayCover/PlayCover/Model/ PlayApp.swift:64-116	PlayCover 同步设置、检查 PlayTools、 安装插件并启动目标应用。
[L]	PlayCover/PlayCover/Model/ PlayApp.swift:124-163	清除 DYLD 和 Metal debug/capture 环 境变量。
[L]	PlayCover/PlayCover/Utils/ PlayTools.swift:46-187	framework 复制、Mach-O 注入、插件复 制和签名。
[L]	PlayCover/PlayCover/Model/ AppSettings.swift:10-32	MaaTools、端口、窗口尺寸和 custom scaler 设置字段。
[L]	PlayCover/PlayCover/Views/ AppSettingsView.swift:576-586	MaaTools 开关和端口 UI。
[L]	PlayCover/PlayCover.xcodeproj/ project.pbxproj:578-635	Carthage framework 复制、Catalyst 转 换、重打包和签名脚本。
[L]	PlayCover/Cartfile:1、 PlayCover/Cartfile.resolved:1	PlayTools 来源和固定 revision。
[X]	PlayTools@7aa1977: MaaTools.swift、 AKPlugin.swift	现有截图经 AKInterface.windowImage, 再走 ScreenCaptureKit/CGWindow; 没 有 direct Metal capture。
[X]	PlayTools@7aa1977: PlayCover.swift、PlayLoader.m	目标进程初始化顺序和 constructor 入口。
[R]	已安装 Arknights UnityFramework	Unity Metal、CAMetalLayer、Unity surface 和 present 相关静态线索。
[R]	已运行目标进程	PlayTools、AKInterface、 UnityFramework、AGXMetal、 IOSurface 和 IOAccelerator 已加载。
[I]	Unity 私有 surface/command-buffer 方 法	可作为当前版本 hook 候选, 但必须通过 selector encoding 和运行时调用验证。

B 当前状态声明

本文是开发方案, 不是功能实现报告。文中所有 M0 之后的 hook、GPU copy、RND0 协议、MaaCore 修改、构建产物和性能数字均为待实施或待验证项目; 当前已完成的是 PlayCover、固定 PlayTools revision、Arknights IPA/安装包和运行时模块的调查。